

COURSE

Основы программирования

JS

Переменные и типы данных языка JavaScript

shesnokov@gmail.com

ОГЛАВЛЕНИЕ

План лекции *3*

Теоретическая часть *3*

Переменные **3**

Ограничения на задание имен переменных **3**

Константы **5**

Правила именования переменных **5**

Типы данных (Число, Строка, Булевый тип, Значение NULL, Значение Undefined, тип Object), примитивные и ссылочные типы, конвертация типов данных **6**

Типы данных 6

Тип данных «Число» («number») 6

Тип данных «Строка» («string») 6

Булевый (логический) тип «boolean» 7

Специальное значение «null» 7

Специальное значение «undefined» 7

Тип данных «Символ» («Symbol») 7

Тип данных «Объект» («object») 8

Оператор TypeOf 8

Объектные оболочки примитивных типов, методы и свойства объекта **9**

Встроенный объект «Number» 9

Встроенный объект «Boolean» 10

Встроенный объект «String» 11

Встроенный объект «Date» 12

Встроенный объект «Array» 15

Встроенный объект «Math» 16

Преобразование типов **17**

Приведение к строке 17

Булевое преобразование 18

Численное преобразование 19

Область видимости переменных **20**

Объект Global **21**

Сборщик мусора **23**

Литература и другие источники *26*

Вопросы для самопроверки *26*

ПЛАН ЛЕКЦИИ

1. Переменные
2. Константы
3. Типы данных
4. Область видимости переменных
5. Объектные оболочки
6. Объект Global
7. Сборка мусора

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Переменные

В зависимости от того, для каких задач Вы создаете скрипт, понадобится работать с различной информацией. Если это электронный магазин – то это товары, корзина. Если чат – посетители, сообщения и так далее. Чтобы хранить информацию, используются ***переменные***.

Переменная — поименованная, либо адресуемая иным способом область памяти, адрес которой можно использовать для осуществления доступа к данным и изменять значение в ходе выполнения программы.

Переменная состоит из ***имени*** и ***выделенной области памяти***, которая ему соответствует. Для ***объявления*** или, другими словами, ***создания переменной*** используется ключевое слово ***`var`***:

```
var message;
```

После объявления, можно записать в переменную данные:

```
var message;
message = 'Hello'; // сохраним в переменной строку
```

Эти данные будут сохранены в соответствующей области памяти и в дальнейшем доступны при обращении по имени:

```
var message;
message = 'Hello!';
console.log(message); // выведет содержимое переменной
```

Для краткости можно совместить объявление переменной и запись данных (***инициализацию переменной***):

```
var message = 'Hello!';
```

Можно даже объявить несколько переменных сразу:

```
var user = 'John', age = 25, message = 'Hello';
```

Ограничения на задание имен переменных

На имя переменной в JavaScript наложены всего два ограничения.

1. Имя может состоять из: букв, цифр, символов `$` и `_`
2. Первый символ не должен быть цифрой.

Например,

```
var myName;
var test123;
```

Что особенно интересно – доллар '$' **и знак подчеркивания** '_' **являются такими же обычными символами, как буквы:**

```
var $ = 1; // объявили переменную с именем '$'
var _ = 2; // переменная с именем '_'
console.log($ + _); // 3
```

А такие переменные были бы неправильными:

```
var 1a; // начало не может быть цифрой
var my-name; // дефис '-' не является разрешенным символом
```

Регистр букв имеет значение. Переменные apple и AppLE – две разные переменные. Русские буквы допустимы, но не рекомендуются. В названии переменных можно использовать и русские буквы, например,

```
var имя = "Вася";
console.log( имя ); // "Вася"
```

Технически, ошибки здесь нет, но на практике сложилась традиция использовать в именах только английские буквы.

Существует список зарезервированных слов, которые нельзя использовать для переменных, так как они используются самим языком, например, var, class, return, export и др. Например, такая запись выдаст синтаксическую ошибку:

```
var return = 5; // ошибка
console.log(return);
```

В старом стандарте JavaScript разрешалось создавать переменную и без var, просто присвоив ей значение:

```
num = 5; // переменная num будет создана, если ее не было
```

В режиме ***"use strict"*** так делать уже нельзя. Следующий код выдаст ошибку:

```
"use strict";
num = 5; // error: num is not defined
```

Обратим внимание, директиву use strict нужно ставить до кода, иначе она не сработает:

```
var something;
"use strict"; // слишком поздно
num = 5; // ошибки не будет, так как строгий режим не активирован
```

Константы

Константа – это переменная, которая никогда не меняется. Вы можете создать именованную константу, доступную только для чтения, используя ключевое слово ***const*** (нововведение стандарта ECMAScript 6). Синтаксис идентификатора константы является таким же, как и у идентификатора переменной: он должен начинаться с буквы, нижнего подчеркивания или знака $ и может содержать буквы, цифры или нижнее подчеркивание. Например,

```
const COLOR_RED = "#F00";
const COLOR_GREEN = "#0F0";
const COLOR_BLUE = "#00F";
const COLOR_ORANGE = "#FF7F00";
const color = COLOR_ORANGE;
console.log(color); // #FF7F00
```

Нельзя изменить значение константы через присваивание или повторное объявление во время выполнения скрипта. Значение должно быть указано при инициализации.

Правила, касающиеся области видимости, для констант являются такими же, как и для переменных, объявленных через let. Если ключевое слово const не указано, то идентификатор будет являться переменной.

Нельзя объявить константу с таким же именем, как у функции или переменной в одной области видимости.

Правила именования переменных

Существует несколько популярных нотаций именования:

- Верблюжья нотация (camelCase, UpperCamelCase): MyClass
- Змеиная нотация (snake_case, underscore): my_const
- Особняком стоит «Венгерская нотация»: pBoxWindow, szClientName, bIsEmpty.

В реальности их гораздо больше, хотя многие вышли из обихода и не употребляются, либо употребляются крайне редко.

В сообществе устоялись правила, относящиеся к большинству языков. Их нужно знать, чтобы писать код, понятный другим разработчикам. Вот так принято называть переменные в зависимости от содержимого:

- значение — существительное: user, page;
- массив — существительное в множественном числе: users, pages;
- функция — глагол: download, update, open, resize;
- класс — существительное с заглавной буквой: User, Page;
- константа — всё имя пишется заглавными буквами в стиле snake_case: MAXIMUM_USERS, LIMIT_PAGE_X;
- обработчик — имя описывает событие, по которому он вызывается. Как правило, содержит префикс on или постфикс Handler: onButtonClick, formSendHandler.

Использование этих правил сделает ваш код намного понятнее.

Типы данных (Число, Строка, Булевый тип, Значение NULL, Значение Undefined, тип Object), примитивные и ссылочные типы, конвертация типов данных

Типы данных

Стандарт ECMAScript 6 определяет семь типов данных:

- Шесть типов данных, которые являются примитивами:
  - ***Boolean***. true и false.
  - ***null***. Специальное ключевое слово, обозначающее нулевое или «пустое» значение. Поскольку JavaScript чувствителен к регистру, null не то же самое, что Null, NULL или любой другой вариант.
  - ***undefined***. Свойство глобального объекта; переменная, не имеющая присвоенного значения, обладает типом undefined.
  - ***Number***. 42 или 3.14159.
  - ***String***. "Howdy".
  - ***Symbol*** (ECMAScript 6).
- Комплексные (составные) типы:
  - ***Object.***

Тип данных «Число» («number»)

```
var n = 123;
n = 12.345;
```

Единый тип *число* используется как для целых, так и для дробных чисел. Существуют специальные числовые значения ***Infinity (бесконечность)*** и ***NaN (ошибка вычислений)***. Например, бесконечность Infinity получается при делении на ноль:

```
console.log(1 / 0); // Infinity
```

Ошибка вычислений NaN будет результатом некорректной математической операции, например,

```
console.log("не число" * 2); // NaN, ошибка
```

Эти значения формально принадлежат типу «число», хотя, конечно, числами в их обычном понимании не являются.

Тип данных «Строка» («string»)

```
var str = "Текст в двойных кавычках";
str = 'Одинарные кавычки тоже подойдут';
```

В JavaScript ***одинарные и двойные кавычки равноправны***. Можно использовать или те или другие.

Тип символ не существует, есть только строка. В некоторых языках программирования есть специальный тип данных для одного символа. Например, в языке C это char. В JavaScript есть только тип «строка» string. Что, надо сказать, вполне удобно.

Булевый (логический) тип «boolean»

У него всего два значения: true (истина) и false (ложь). Как правило, такой тип используется для хранения значения типа да/нет, например,

```
var checked = true; // поле формы помечено галочкой
checked = false; // поле формы не содержит галочки
```

Специальное значение «null»

Значение null не относится ни к одному из типов выше, а образует свой отдельный тип, состоящий из единственного значения null:

```
var age = null;
```

В JavaScript null не является «ссылкой на несуществующий объект» или «нулевым указателем», как в некоторых других языках. Это просто специальное значение, которое имеет смысл «ничего» или «значение неизвестно». В частности, код выше говорит о том, что возраст age неизвестен.

Специальное значение «undefined»

Значение undefined, как и null, образует свой собственный тип, состоящий из одного этого значения. Оно имеет смысл «значение не присвоено». Если переменная объявлена, но в неё ничего не записано, то её значение как раз и есть undefined:

```
var x;
console.log(x); // выведет "undefined"
```

Можно присвоить undefined и в явном виде, хотя это делается редко:

```
var x = 123;
x = undefined;
console.log(x); // "undefined"
```

В явном виде undefined обычно не присваивают, так как это противоречит его смыслу. Для записи в переменную «пустого» или «неизвестного» значения используется null.

Тип данных «Символ» («Symbol»)

Новый примитивный тип данных ***Symbol*** служит для создания уникальных идентификаторов. Например,

```
'use strict';
let sym = Symbol();
console.log( typeof sym ); // symbol
```

Каждый символ – уникален. У функции Symbol есть необязательный аргумент «имя символа». Его можно использовать для описания символа, в целях отладки, например,

```
'use strict';
let sym = Symbol("name");
console.log( sym.toString() ); // Symbol(name)
```

Но при этом, если у двух символов одинаковое имя, то это не значит, что они равны:

```
console.log( Symbol("name") == Symbol("name") ); // false
```

Если хочется из разных частей программы использовать именно одинаковый символ, то можно передавать между ними объект символа или же – использовать ***«глобальные символы»*** и ***«реестр глобальных символов»***.

Тип данных «Объект» («object»)

Объект — это структура данных (целый их набор), представленный в виде набора пар «ключ-значение». Значением может быть любой из типов данных — тогда это будет свойство объекта, или даже функция — тогда это будет метод объекта. Тип данных ‘object’ используется для создания коллекций данных и для объявления более сложных сущностей. Объявляются объекты при помощи фигурных скобок {...}, например,

```
var user = {name: "Вася"};
```

Оператор TypeOf

Оператор typeof возвращает тип аргумента. У него есть два синтаксиса: со скобками и без:

- Синтаксис оператора: typeof x.
- Синтаксис функции: typeof(x).

Работают они одинаково, но первый синтаксис короче. Результатом typeof является строка, содержащая тип:

```
console.log(typeof undefined); // "undefined"
console.log(typeof 0); // "number"
console.log(typeof true); // "boolean"
console.log(typeof "foo"); // "string"
console.log(typeof {}); // "object"
console.log(typeof null)); // "object"  (1)
console.log(typeof function(){}); // "function"  (2)
```

Последние две строки помечены, потому что typeof ведет себя в них по-особому.

1. Результат typeof null == "object" – это официально признанная ошибка в языке, которая сохраняется для совместимости. На самом деле null – это не объект, а отдельный тип данных.
2. Функции будут рассмотрены чуть позже. Пока можно заметить, что функции не являются отдельным базовым типом в JavaScript, а подвидом объектов. Но typeof выделяет функции отдельно, возвращая для них "function". На практике это весьма удобно, так как позволяет легко определить функцию.

Объектные оболочки примитивных типов, методы и свойства объекта

Все языки программирования имеют ***внутренние (встроенные) объекты***, реализующие основную функциональность языка. Встроенные объекты являются фундаментом языка, на котором строится специализированный код, реализующий функциональность, продиктованную вашим воображением. В JavaScript есть много встроенных объектов, которые в значительной степени определяют возможности языка.

Встроенный объект «Number»

JavaScript-объект Number является ***оболочкой*** для числовых значений. Его можно использовать вместе с ключевым словом new и присваивать переменной для дальнейшего использования в JavaScript-коде. Например,

```
var myNumber = new Number(numeric value);
```

Другой способ создания объекта Number – простое присваивание переменной числового значения. Эта переменная будет иметь доступ к свойствам и методам объекта.

Помимо собственно возможности хранения числовых значений, объект Number имеет ***свойства*** и ***методы*** для манипулирования числами и извлечения информации о них. Все свойства являются ***константами***, доступными только для чтения, т.е. их значения всегда остаются неизменными. Объект Number имеет четыре свойства:

MAX_VALUE
MIN_VALUE
NEGATIVE_INFINITY
POSITIVE_INFINITY

MAX_VALUE возвращает значение 1.7976931348623157e+308, являющееся самым большим числом, которое может обрабатывать JavaScript. Например,

```
console.log(Number.MAX_VALUE);
// Результат: 1.7976931348623157e+308
```

MIN_VALUE возвращает значение 5e-324, являющееся самым маленьким числом, возможным в JavaScript:

```
console.log(Number.MIN_VALUE);
// Результат: 5e-324
```

NEGATIVE_INFINITY – это значение, меньшее любого отрицательного числа, которое может обработать JavaScript (представляется как -Infinity):

```
console.log(Number.NEGATIVE_INFINITY);
// Результат: -Infinity
```

POSITIVE_INFINITY – это значение, большее, чем MAX_VALUE (представляется как Infinity):

```
console.log(Number.POSITIVE_INFINITY);
// Результат: Infinity
```

Объект Number также имеет методы, служащие для форматирования или преобразования числовых значений. Такими методами являются:

toExponential
toFixed
toPrecision
toString
valueOf

Каждый метод, по существу, делает в точности то, на что указывает его название, – например, ***метод toExponential*** возвращает строковое представление числа в экспоненциальной форме. Уникальность каждого метода заключается в принимаемых им аргументах. ***Метод toExponential*** имеет необязательный аргумент, который используется для настройки количества значащих цифр, ***метод toFixed*** на основании переданного аргумента выбирает точность в знаках после десятичной запятой, а ***метод toPrecision*** на основании аргумента выбирает число отображаемых значащих цифр.

Методы toString и ***valueOf*** имеют все объекты JavaScript. ***Метод toString*** возвращает строковое представление числа (в данном случае), но для других объектов он возвращает строковое представление их объектного типа. ***Метод valueOf*** возвращает примитивное значение объектного типа, который его вызвал; в данном случае это объект Number.

Сам по себе объект Number не выглядит особенно мощным, но он является важной частью любого языка программирования, и JavaScript не исключение. JavaScript-объект Number предоставляет фундамент для любых математических процедур, являющихся по существу основой всех языков программирования.

Встроенный объект «Boolean»

Объект Boolean необходим при создании любой логики в JavaScript. ***Boolean*** – это объект, представляющий значение true (истина) или false (ложь). Объект Boolean имеет несколько значений, интерпретируемых как "ложь": (0, -0, null, "" [пустая строка]), неопределенное значение (NaN), и, естественно, false. Все другие значения Boolean интерпретируются как true. Экземпляр объекта можно создать при помощи ключевого слова new, но чаще применяется присваивание переменной значения true или false:

```
var myBoolean = new Boolean(true);
```

Объект Boolean имеет методы toString и valueOf, хотя, скорее всего, вам не придется их использовать. Объект Boolean чаще всего используется для оценки истинности значений в условных операторах. Сочетание объектов Boolean и условных операторов позволяет создавать логику с помощью JavaScript. Примерами таких условных операторов являются if, if...else, if...else...if и switch. Объекты Boolean используются в сочетании с условным оператором для определения результата, соответствующего заданным условиям. В листинге приведен пример условного оператора в сочетании с Boolean.

```
var myBoolean = true;
if(myBoolean == true) {
  // Если условие истинно
} else {
  // Если условие ложно
}
```

Не стоит даже говорить, насколько важной частью JavaScript является объект Boolean. Без него в условных операторах нечего было бы сравнивать.

Встроенный объект «String»

JavaScript-объект String является оболочкой для текстовых значений. Помимо собственно возможности хранения текстовых данных, объект String имеет свойства и методы для манипулирования или сбора информации о текстовых данных. Как и в случае с объектом Boolean, объект String создавать не обязательно – можно, например, присвоить переменной строковое значение, и ей станут доступны все свойства и методы объекта String:

```
var myString = new String("My string");
```

Объект String имеет всего одно свойство, length (длина), доступное только для чтения. Свойство length используется для возврата длины строки и не может быть изменено извне. Ниже приведен пример использования свойства length для определения количества символов в строке:

```
console.log(myString.length);
// Результатом является числовое значение 9
```

В результате этот код выдает значение 9, поскольку пробелы между двумя словами также учитываются при подсчете.

В объекте String есть несколько методов, которые используются для манипулирования текстовыми данными и сбора информации о них. Ниже приведен перечень доступных методов:

charAt
charCodeAt
concat
fromCharCode
indexOf
lastIndexOf
match
replace
search
slice
split
substr
substring
toLowerCase
toUpperCase

Метод chartAt можно использовать для извлечения конкретного символа по индексу, переданному в качестве аргумента. Приведенный ниже пример кода возвращает первый символ строки:

```
var myString = "My string";
console.log(myString.chartAt(0));
```

```
// Результат - M
```

Если необходим противоположный результат, существует пара методов, возвращающих индекс указанного символа или набора символов в строке. Это методы indexOf и lastIndexOf, которые принимают два параметра: searchString и start. Параметр searchString является начальным индексом, а параметр start указывает, где нужно начинать поиск. Разница между этими методами состоит в том, что indexOf возвращает первое появление в строке, а lastIndexOf – последнее.

Метод charCodeAt похож на charAt. Единственное отличие состоит в том, что он возвращает символ в кодировке Unicode. Еще одним методом объекта String, работающим с Unicode, является fromCharCode, преобразующий Unicode в символы.

Если нужно объединить строки, можно либо соединить их при помощи знака плюс (+), либо соответствующим образом использовать метод concat. Этот метод принимает неограниченное число строковых аргументов, соединяет их и возвращает результат в виде новой строки. В листинге 2 приведен пример объединения несколько строк в одну при помощи экземпляра concat.

```
var myString1 = "My";
var myString2 = " ";
var myString3 = "string";
console.log(myString.concat(myString1, myString2, myString3);
// Результат - "My String"
```

Существует также группа методов String, принимающих в качестве аргументов регулярные выражения для поиска или изменения строк. К ним относятся методы match, replace и search. Метод match использует регулярное выражение для поиска определенной строки и возвращает все совпадения. Метод replace принимает подстроку либо регулярное выражение и замещающую строку (в качестве второго параметра) и возвращает обновленную строку. Последний из этих методов, search, ищет совпадение с регулярным выражением и возвращает его позицию.

Для изменения строк существует несколько методов. Первым является метод slice, который извлекает и возвращает часть строки по заданному индексу или по комбинации начального и конечного индексов. Другим таким методом является метод split. Он разбивает строку на массив подстрок там, где обнаруживает символ-разделитель. Если, например, в качестве аргумента передан знак "запятая" (,), строка будет разделена на новые подстроки по каждой запятой. Менять строки могут также метод substr, принимающий в качестве аргументов начальную позицию и количество символов и извлекающий из строки подстроку соответствующей длины начиная с указанной позиции, и метод substring, извлекающий подстроку по начальному и конечному индексам, переданным в качестве аргументов. И, наконец, имеются методы toLowerCase и toUpperCase, преобразующие символы строки в нижний и верхний регистр соответственно. Эти методы полезны при сравнении значений строк, поскольку строки иногда могут записываться в разных регистрах. Эти методы позволяют сравнить значения без учета регистра символов.

Встроенный объект «Date»

JavaScript-объект Date позволяет работать с датами и временем. Этот объект можно создать разными путями в зависимости от требуемых результатов. Например, можно создать объект без аргументов:

```
var myDate = new Date();
```

Можно также в качестве аргумента передать milliseconds, например,

```
var myDate = new Date(milliseconds);
```

Можно передать в качестве аргумента строку с датой, например,

```
var myDate = new Date(dateString);
```

Можно передать несколько аргументов, представляющих составные части даты, например,

```
var myDate = new Date(year, month, day, hours, minutes, seconds, milliseconds);
```

Кроме того, для объекта Data доступны несколько методов, которые можно использовать сразу после создания объекта. Большинство доступных методов предназначено для получения определенных частей текущего времени. Для объекта Data доступны следующие getter-методы:

getDate
getDay
getFullYear
getHours
getMilliseconds
getMinutes
getMonth
getSeconds
getTime
getTimezoneOffset

Как можно заметить, каждый метод довольно прост в плане возвращаемых им значений. Разница состоит в диапазоне возвращаемых значений. Например, ***метод getDate*** возвращает номер дня месяца (от 1 до 31), ***метод getDay*** возвращает номер дня недели (от 0 до 6), ***метод getHours*** возвращает числовое значение часа (от 0 до 23), а ***функция getMilliseconds*** возвращает числовое значение миллисекунд (от 0 до 999). ***Методы getMinutes*** и ***getSeconds*** возвращают значения в диапазоне от 0 до 59, а ***метод getMonth*** возвращает номер месяца (от 0 до 11). Особняком в этом списке стоят только ***методы getTime*** и ***getTimezoneOffset***. ***Метод getTime*** возвращает число миллисекунд, прошедших с полуночи 1 января 1970 года, а ***метод getTimezoneOffset*** возвращает разницу в минутах между временем по Гринвичу и локальным временем.

Для большинства ***getter-методов*** существуют ***setter-методы***, принимающие числовой аргумент соответствующего диапазона значений. Вот эти методы:

setDate
setFullYear
setHours
setMilliseconds
setMinutes
setMonth
setSeconds
setTime

Для всех перечисленных выше getter-методов существуют аналогичные методы, возвращающие те же диапазоны значений, но только по Гринвичу. Вот эти методы:

getUTCDate
getUTCDay
getUTCFullYear
getUTCHours
getUTCMilliseconds
getUTCMinutes
getUTCMonth
getUTCSeconds

Естественно, поскольку для всех оригинальных getter-методов существуют setter-методы, есть их аналоги, использующие время по Гринвичу. Вот эти методы:

setUTCDate
setUTCFullYear
setUTCHours
setUTCMilliseconds
setUTCMinutes
setUTCMonth
setUTCSeconds

В объекте Date есть много методов, преобразующих даты в строки, о которых стоит упомянуть. В некоторых ситуациях необходимо преобразовать дату или часть даты в строку – например, если дата добавляется к строке или используется в операторе сравнения. Для объекта Date доступно несколько методов, немного по-разному преобразующих даты в строки, в том числе:

toDateString
toLocaleDateString
toLocaleTimeString
toLocaleString
toTimeString
toUTCString

Метод toDateString преобразует дату в строку (текущая дата возвращается в формате Wed Jul 29 2020), например,

```
var myDate = new Date();
console.log(myDate.toDateString());
```

Метод toTimeString преобразует время из объекта Date в строку (время возвращается в формате 15:52:49 GMT+0300 (Москва, стандартное время)), например,

```
var myDate = new Date();
console.log(myDate.toTimeString());
```

Последним методом, преобразующим дату в строку, является toUTCString, который работает с временем по Гринвичу.

В JavaScript-объект Date на первый взгляд кажется простым, но он может быть не просто полезным способом отобразить текущую дату. Все зависит от функциональности, которую необходимо создать. Например, объект Date может быть основой для создания счетчика обратного времени или любой другой функциональности, связанной со временем.

Встроенный объект «Array»

JavaScript-объект ***Array (массив)*** – это «накаченная» переменная. Он используется для хранения нескольких значений в одной переменной одновременно и имеет много методов, позволяющих манипулировать хранящимися в нем значениями и собирать информацию о них. Хотя объект Array не ограничивает типы значений, рекомендуется использовать во всем массиве значения одинакового типа. Иначе говоря, не рекомендуется использовать числа и строки в одном и том же массиве. Все свойства объекта Array доступны только для чтения, т.е. их нельзя изменить извне.

Единственным свойством, доступным для объекта Array, является length (длина). Это свойство возвращает количество элементов в массиве и часто используется при итерировании по значениям массива в цикле:

```
var myArray = new Array(1, 2, 3);
for(var i=0; i<myArray.length; i++) {
  console.log(myArray[i]);
}
```

Для объекта Array доступно несколько методов, которые можно использовать для добавления и удаления элементов массива. К этим методам относятся ***pop***, ***push***, ***shift*** и ***unshift***. Методы pop и shift удаляют элементы из массива. Метод pop удаляет и возвращает последний элемент в массиве, тогда как метод shift удаляет и возвращает первый элемент в массиве. Противоположную функциональность реализуют методы push и unshift, которые добавляют элементы в массив. Метод push добавляет новые элементы в конец массива и возвращает новую длину, а метод unshift добавляет элементы в начало массива и возвращает новую длину.

Сортировку массива в JavaScript можно выполнить при помощи двух методов, один из которых называется ***sort***. Другой называется ***reverse***. Сложность метода sort заключается в том, что он сортирует массив, основываясь на необязательной функции sort. Функцией sort может быть любая написанная вами специализированная функция. Метод reverse не так сложен, как sort, хотя он изменяет порядок элементов в массиве – расставляет элементы в обратном порядке.

При работе с массивами активно используются индексы, которые определяют местоположение каждого элемента в массиве. Есть два метода изменения массива, основанных на индексах: ***slice*** и ***splice***. Метод slice принимает индекс или пару из начального и конечного индексов в качестве аргументов, а затем на их основе извлекает и возвращает часть массива в виде нового массива. Метод splice принимает параметры index, length и unlimited. Метод добавляет элементы в массив, удаляет элементы из массива или одновременно добавляет и удаляет элементы, исходя из указанного индекса. Существует также метод ***indexOf***, который может возвращает индекс по найденному значению. Этот индекс можно использовать для выполнения методов slice или splice.

Для любого языка программирования справедливо утверждение, что хорошо написанный код – это хорошо организованный код. Как показывает наше описание различных методов, JavaScript-объект Array является мощным средством организации данных и создания сложной функциональности.

Встроенный объект «Math»

JavaScript-объект Math используется для выполнения математических функций. Создать экземпляр этого объекта нельзя; вы просто используете объект Math как есть и вызываете его свойства и методы без создания экземпляра:

```
var pi = Math.PI;
```

Объект Math имеет много свойств и методов, реализующих математическую функциональность JavaScript. Все свойства Math являются константами с доступом только по чтению. Вот их список:

E
LN2
LN10
LOG2E
LOG10E
PI
SQRT1_2
SQRT2

Свойство ***E*** возвращает основание натуральных логарифмов («число Эйлера»), равное 2.718281828459045. Два других свойства, ***LN2*** и ***LN10***, используются для вычисления натуральных логарифмов. Свойство LN2 возвращает натуральный логарифм числа 2, тогда как свойство LN10 возвращает натуральны логарифм числа 10. Свойства ***LOG2E*** и ***LOG10E*** используются для вычисления логарифма E по основанию 2 или 10. Результат LOG2E равен 1.4426950408889633, а результат LOG10E равен 0.4342944819032518. Метод ***PI*** возвращает отношение длины окружности к ее диаметру. Значения квадратного корня возвращают два свойства: ***SQRT1_2*** и ***SQRT2***. Первое свойство возвращает квадратный корень из 0.5, а второе – квадратный корень из 2.

Кроме этих свойств, есть несколько методов, которые можно использовать для возврата различных числовых значений. Каждый из этих методов принимает числовое значение и возвращает значение в зависимости от названия метода. К сожалению, названия методов не всегда самоочевидны:

abs (абсолютное значение числа)
acos (арккосинус)
asin (арксинус)
atan (арктангенс)
atan2 (арктангенс, зависящий от двух аргументов)
cos (косинус)
exp (экспонента)
log (натуральный логарифм числа)
pow (значение x в степени y)
sin (синус)
sqrt (квадратный корень)
tan (тангенс угла)

Для округления чисел в JavaScript используются три метода: ***ceil***, ***floor*** и ***round***. Метод ceil возвращает число, округленное до ближайшего большего значения. Этот метод полезен, когда необходимо округлить число в большую сторону до ближайшего целого значения. Метод floor имеет противоположную функциональность: он возвращает число, округленное до ближайшего меньшего значения. Этот метод полезен, когда необходимо округлить число в меньшую сторону до ближайшего целого значения. Метод round предоставляет обычную функциональность округления, увеличивая или уменьшая число в зависимости от имеющихся десятичных знаков.

Последними тремя методами объекта Math являются ***max***, ***min*** и ***random***. Метод max принимает несколько числовых аргументов и возвращает наибольшее значение, тогда как метод min возвращает наименьшее значение. Эти методы могут быть полезны при сравнении переменных с числовыми значениями, особенно когда эти числовые значения неизвестны. Метод random возвращает случайное число в диапазоне от 0 до 1.

Преобразование типов

JavaScript — это ***динамически типизированный язык***. Это означает, что вам не нужно указывать тип данных переменной, когда вы ее объявляете, типы данных преобразуются автоматически по мере необходимости во время выполнения скрипта. Так, например, вы можете определить переменную следующим образом:

```
var answer = 42;
```

А позже вы можете присвоить этой переменной строковое значение, например,

```
answer = "Thanks for all the fish...";
```

Поскольку JavaScript является динамически типизированным, это присваивание не вызовет сообщения об ошибке.

Когда разработчик хочет намеренно произвести преобразование типов, написав, например, `Number(value)`, это называется **явным преобразованием типов** (или type casting).

Так как JavaScript это слабо типизированный язык, преобразование между разными типами может происходить автоматически, и это называется **неявным преобразованием типов.** Чаще всего это происходит когда вы применяете операторы к значениям разных типов, таких как `1 == null`, `2 / `5``, `null + new Date()`, может происходить в зависимости от контекста, как например, в случае с `if (value) {…}`, где value будет приведено к булевому значению.

В JavaScript существует всего 3 типа преобразования:

- строковое
- булевое
- численное

Логика преобразования для примитивов и объектов работает по-разному, но, и примитивы и объекты могут быть преобразованы только этими тремя способами.

Приведение к строке

Для явного приведения значения к строке необходимо применить к нему функцию String(). Неявное преобразование будет вызвано бинарным оператором +, кода один из операндов является строкой:

```
String(123) // explicit
123 + ''    // implicit
```

Все примитивы будут приведены к строке вполне естественно, как вы могли и ожидать:

```
String(123)             // '123'
String(-12.3)           // '-12.3'
String(null)            // 'null'
String(undefined)       // 'undefined'
String(true)            // 'true'
String(false)           // 'false'
```

Преобразование символов происходит немного сложнее, потому что они могут быть преобразованы только явным образом.

```
String(Symbol('my symbol'))   // 'Symbol(my symbol)'
'' + Symbol('my symbol')      // TypeError is thrown
```

Булевое преобразование

Для явного преобразования к булевому значению, нужно применить функцию Boolean(). Неявное преобразование происходит в логическом контексте if (val) { … } или при применении логических операторов (|| && !).

```
Boolean(2)        // explicit
if (2) { ... }    // implicit due to logical context
!!2               // implicit due to logical operator
2 || 'hello'      // implicit due to logical operator
```

На заметку, логические операторы такие как || и && производят булевое преобразование под капотом, но при этом всегда возвращают оригинальное значение операндов, даже если они не являются булевыми.

```
// returns number 123, instead of returning true
// 'hello' and 123 are still coerced to boolean internally to calculate the expression
let x = 'hello' && 123;   // x === 123
```

Поскольку существует всего два возможных результата преобразования, легче просто запомнить список ложных значений:

```
Boolean('')         // false
Boolean(0)          // false
Boolean(-0)         // false
Boolean(NaN)        // false
Boolean(null)       // false
Boolean(undefined)  // false
Boolean(false)      // false
```

Любое значение, которое не вошло в этот список, будет преобразовано в true, включая объекты, функции, Array, Date и так далее. Символы, пустые объекты и массивы так же будут иметь значение true.

```
Boolean({})          // true
Boolean([])          // true
Boolean(Symbol())    // true
!!Symbol()           // true
Boolean(function() {})  // true
```

Численное преобразование

Для явного преобразования к числу нужно применить функцию Number(), точно так же, как мы делали с Boolean() и String().

Неявное преобразование несколько запутаннее, так как оно вызывается в большем количестве случаев.

- операторы сравнения (>, <, <=,>=)
- бинарные операторы (| & ^ ~)
- арифметические операторы (- + * / %). Обратите внимание, что бинарный оператор + не вызывает численного преобразования, если один из операндов является строкой
- унарный оператор +
- оператор нестрогого равенства == (включая !=). Обратите внимание, что данный оператор не вызывает численное преобразование, если оба операнда являются строками

```
Number('123')   // explicit
+'123'          // implicit
123 != '456'    // implicit
4 > '5'         // implicit
5/null          // implicit
true | 0        // implicit
```

Примеры того, как примитивы будут преобразованы в числа:

```
Number(null)              // 0
Number(undefined)         // NaN
Number(true)              // 1
Number(false)             // 0
Number(" 12 ")            // 12
Number("-12.34")          // -12.34
Number("\n")              // 0
Number(" 12s ")           // NaN
Number(123)               // 123
```

При преобразовании строки в число, движок сначала отсекает все пробельные символы, символы \n, и \t в начале и в конце строки, и возвращает NaN если обрезанная строка не представляет из себя корректное число. Если строка окажется пустой, то результатом будет 0.

null и undefined обрабатываются по разному: null станет 0, в то время как undefined станет NaN.

Численное преобразование, как явное, так и неявное, не работает для символов. Более того, движок бросает ошибку TypeError, вместо того, чтобы по-тихому преобразовать Symbol в NaN, как это происходит с undefined.

```
Number(Symbol('my symbol'))    // TypeError is thrown
+Symbol('123')                 // TypeError is thrown
```

Существует два **специальных правила** которые следует запомнить:

1. При применении == к null или undefined, численное преобразование не происходит, так как null может равняться только null или undefined, и ничему другому.

```
null == 0            // false, null is not converted to 0
null == null         // true
undefined == undefined  // true
null == undefined      // true
```

2. `NaN` не равен ничему, даже самому себе.

```
if (value !== value) { console.log("we're dealing with NaN here") }
```

Область видимости переменных

Когда вы объявляете переменную вне функции, то такая переменная называется ***глобальной переменной***, т.к. доступна любому коду в текущем документе. Когда вы объявляете переменную внутри функции, то такая переменная называется ***локальной переменной***, т.к. доступна только внутри данной функции.

До ECMAScript 6 в JavaScript отсутствовала область видимости блока; переменная, объявленная внутри блока, является локальной для функции (или глобальной области видимости), внутри которой находится данный блок. Например, следующий код выведет значение 5, т.к. областью видимости переменной x является функция (или глобальный контекст), внутри которой объявлена переменная x, а не блок, которым в данном случае является оператор if:

```
if (true) {
  var x = 5;
}
console.log(x); // 5
```

Такое поведение меняется, если используется оператор ***let***, введенный в ECMAScript 6:

```
if (true) {
  let y = 5;
}
console.log(y); // ReferenceError
```

У объявлений переменной через let есть одно из основных отличий от var: переменная, объявленная через var, видна везде в функции, а переменная, объявленная через let, видна только в рамках блока {...}, в котором объявлена.

Объект Global

Объект ***Global*** является сервисным объектом JavaScript, который содержит глобальные константы и функции (методы). Одной из особенностей использования данного объекта является то, что его нельзя создавать. Обращение к константам и функциям данного объекта происходит напрямую, т.е. без использования имени объекта и точки.

С константами объекта Global (Infinity, NaN, undefined) мы познакомились на предыдущих уроках, тогда мы ещё не знали, что они являются константами какого-то объекта.

На этом уроке более подробно остановимся на рассмотрении основных функций (методов) объекта Global.

Метод parseInt() предназначен для преобразования строки, указанной в качестве параметра в число. Также позволяет преобразовывать строку, содержащую не только цифры в число, в том случае, если эти цифры расположены в начале.

Например:

```
var str1 = "1200";
var str2 = "200px";
var str3 = "article";
var number1 = parseInt(str1);
var number2 = parseInt(str2);
var number3 = parseInt(str3);
document.write("Переменная number1 = " + number1);
document.write("<br>");
document.write("Переменная number2 = " + number2);
document.write("<br>");
document.write("Переменная number3 = " + number3);
```

```
Переменная number1 = 1200
Переменная number2 = 200
Переменная number3 = NaN
```

Метод parseFloat() преобразует строку, указанную в качестве параметра в число с плавающей точкой. Также позволяет преобразовывать строку, содержащую не только цифры в число, в том случае, если эти цифры расположены в начале.

Например:

```
var str1 = "333.777";
var str2 = "78.43px";
var str3 = "article";
var number1 = parseFloat(str1);
var number2 = parseFloat(str2);
var number3 = parseFloat(str3);
```

```
document.write("Переменная number1 = " + number1);
document.write("<br>");
document.write("Переменная number2 = " + number2);
document.write("<br>");
document.write("Переменная number3 = " + number3);
```

```
Переменная number1 = 333.777
Переменная number2 = 78.43
Переменная number3 = NaN
```

Метод isNaN() возвращает true, если это число равно NaN. В остальных случаем данный метод возвращает значение false.

Например:

```
var str1 = "article";
var number1 = isNaN(0/0);
var number2 = isNaN(10);
var number3 = isNaN(str1);
document.write("Переменная number1 = " + number1);
document.write("<br>");
document.write("Переменная number2 = " + number2);
document.write("<br>");
document.write("Переменная number3 = " + number3);
```

```
Переменная number1 = true
Переменная number2 = false
Переменная number3 = true
```

Метод isFinite() возвращает true, если указанное в качестве параметра значения является конечным числом. В остальных случаях данный метод возвращает значение false.

Например:

```
var str1 = "article";
var number1 = isFinite(-5/0);
var number2 = isFinite(7.87);
var number3 = isFinite(str1);
document.write("Переменная number1 = " + number1);
document.write("<br>");
document.write("Переменная number2 = " + number2);
document.write("<br>");
document.write("Переменная number3 = " + number3);
```

```
Переменная number1 = false
Переменная number2 = true
Переменная number3 = false
```

Методы encodeURI() и decodeURI() предназначенные для кодирования и декодирования универсального идентификатора ресурса (URI). Принцип работы данных методов основан на замещении некоторых символов.

Например:

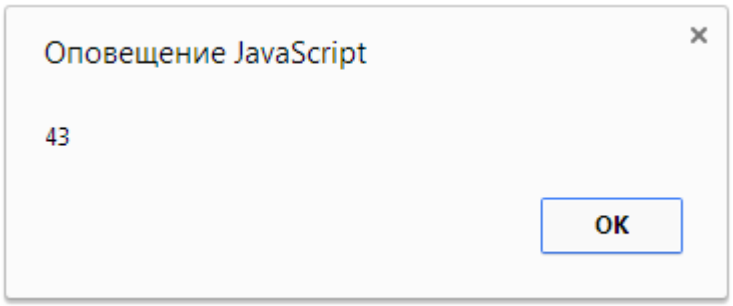
```
var aURL = encodeURI("http://www.mysite.ru/статьи/");
document.write("Encoded URI: "+ aURL);
document.write("<br>");
document.writeln("Decoded URI: "+ decodeURI(aURL));
```

```
Encoded URI: http://www.mysite.ru/%D1%81%D1%82%D0%B0%D1%82%D1%8C%D0%B8/
Decoded URI: http://www.mysite.ru/статьи/
```

Метод eval() рассматривает строчку, указанную в качестве параметра как код на языке JavaScript и пытается его выполнить. Данный метод при написании сценариев использовать не рекомендуется.

Например:

```
var str1 = "alert(16+27);";
eval(str1);
```



Сборщик мусора

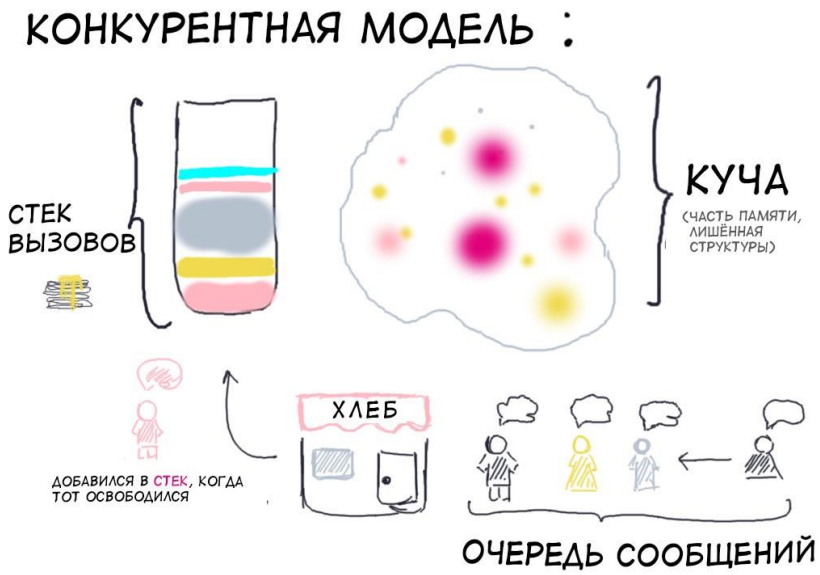
Управление памятью в JavaScript обычно происходит незаметно. Мы создаём примитивы, объекты, функции… Всё это занимает память. Что происходит с объектом, когда он становится «не нужен»? Возможно ли «переполнение» памяти?

Важнейшей задачей ***сборки мусора*** является возможность управления использованием памяти конкретной программой. Такие языки, как C, обычно могут подключаться к управлению памятью программы и выделять, и освобождать её в контексте программы. ECMAScript, с другой стороны, не имеет интерфейса для доступа к управлению памятью (да, это означает отсутствие соответствующего API). Что, в общем-то, означает, что все права на управление памятью в программе передаются браузерному движку V8.

Поскольку у нас нет доступа к бесконечному объему памяти, работа сборщика мусора заключается в том, что нужно перебрать все объекты, для которых выделена память, и определить, «мертвы» они или нет. Те, которые «живы», должны остаться в памяти, те, которые «мертвы», удаляются, а память возвращается обратно в кучу.

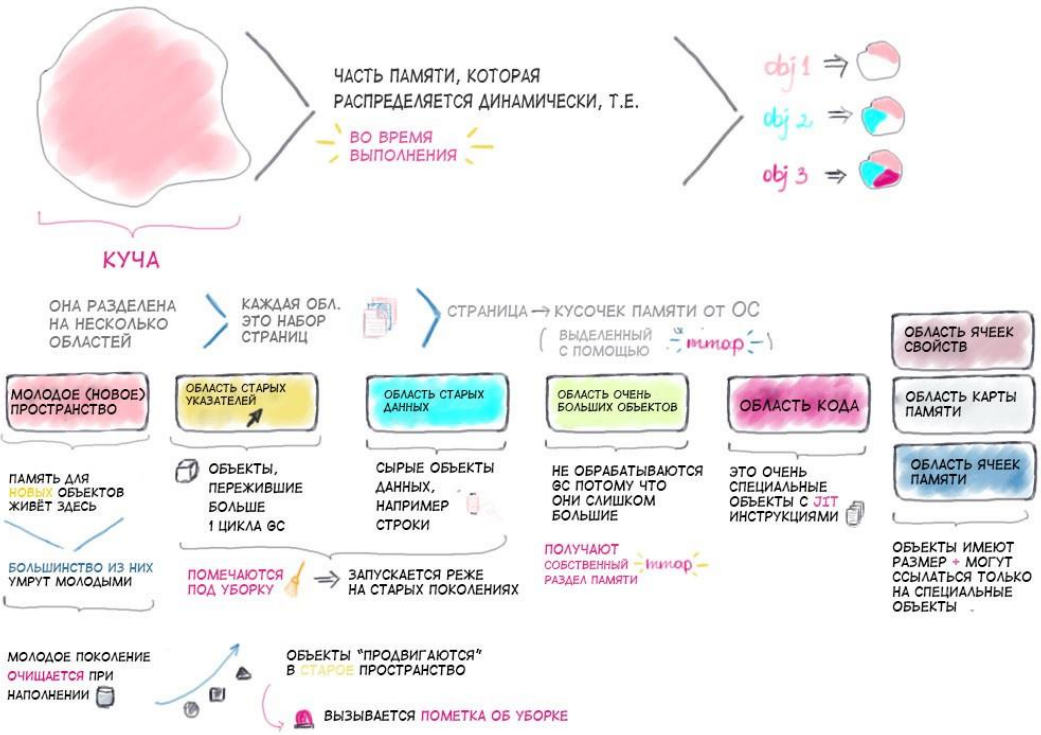
Что такое куча? ***Куча*** — это неструктурированная область, из которой объекты получают выделенную память. Распределение является динамическим, поскольку размер / время жизни / количество объектов неизвестно, поэтому выделение и освобождение памяти происходит во время выполнения

программы. Поэтому, если мы посмотрим на конкурентную модель, куча работает непосредственно со стеком вызовов, так как объекты, отображаемые в стеке, требуют выделения памяти.



Базовая проверка того, «жив» или «мертв» объект, основана на том, может ли клиент или программа, выполняющая код, достичь его. Наиболее достижимый объект, как вы можете подумать, это, вероятно, объект, объявленный в ***корневой области видимости***. О достижимости можно также думать так: сможет ли другой объект или корневой объект получить его, и если да, то память, требуемая для этого объекта, сохраняется.

V8 выделяет память в куче при создании новых объектов или новых «указателей» (в javascript нет настоящих указателей, поэтому «указатели» технически просто копируют ссылки на исходный объект). В куче есть множество разных пространств для разных типов объектов, и она будет организована примерно так:



В целях сбора мусора V8 делит кучу на две части: молодое и старое пространство. Когда вы выполняете операции, требующие выделения памяти, V8 выделяет место в первой части. Когда вы продолжаете добавлять вещи в кучу, вы в конечном итоге исчерпываете память, поэтому V8 будет запускать очистку. Для вновь созданных объектов память выделяется очень быстро из короткой и быстрой коллекции, очищаемой на регулярной основе (для удаления уже мертвых объектов). Как только объекты «переживают» несколько (2, чтобы быть точными) циклов очистки, они переносятся в старое пространство, обрабатывающееся сборщиком мусора в отдельном цикле по факту заполнения.



Старые объекты — это те, которые пережили более одного цикла сборки мусора, то есть на них продолжают ссылаться другие объекты, и они должны оставаться в памяти. Обычно они не ссылаются на молодые объекты, но продолжают иметь ссылки на более старые объекты. Нежелательно допускать смешение поколений, так как это делает уборку менее чистой.

ЛИТЕРАТУРА И ДРУГИЕ ИСТОЧНИКИ

Перевод стандарта ESMAScript 5 http://es5.javascript.ru/

Печатное издание где рассматриваются нововведения стандарта ESMAScript 6: Симпсон К. ES6 и не только. – СПб.: Питер, 2017. – 336 с. (ссылка Google Books)

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение понятию переменная. В чем отличие объявления переменной и ее инициализации?
2. Какие ограничения в JavaScript приняты на именование переменных?
3. В чем отличие константы от обычной переменной (начиная со стандарта ESMAScript 6)?
4. Какие типы данных введены в JavaScript?
5. Как в JavaScript происходит преобразование типов?
6. Варианты использования оператора typeof для определения типа.
7. На что влияет область видимости переменной? Как в JavaScript можно изменить область видимости?
8. Что такое объектная оболочка (встроенный объект)? Перечислите основные объектные оболочки и доступные в них функции.
9. Для чего необходима операция сборки мусора?
10. Опишите процесс сборки мусора в JavaScript.